Code and command output

A retry policy is easier to review when its implementation and observed behavior are kept together. Source listings carry filenames; terminal sessions describe an action and its output.

retry.cpp

```cpp
#include <algorithm>
#include <chrono>

using namespace std::chrono_literals;

std::chrono::milliseconds retry_delay(unsigned retry) {
    auto delay = 100ms;
    constexpr auto cap = 1600ms;
    while (retry > 0 && delay < cap) {
        delay = std::min(delay * 2, cap);
        --retry;
    }
    return delay;
}
```

Build and inspect

```
› clang++ -std=c++17 retry.cpp -o retry
› ./retry
attempt 1: 100 ms
attempt 2: 200 ms
attempt 3: 400 ms
attempt 4: 800 ms
attempt 5: 1600 ms
```

config/retry.conf

```
initial_delay_ms = 100
maximum_delay_ms = 1600
maximum_retries = 5
```

> **Note**
>
> Listing 1 calculates a delay; it does not sleep or send a request. Keep those effects separate from the policy.

Notes that support the argument

Callouts identify the role of a passage. Their labels and accent colors help a reader distinguish context, explanation, advice, and risk without interrupting the surrounding text.

> **Note**
>
> The initial request is not a retry. A budget of five retries permits up to six attempts in total.

> **Explanation**
>
> Increasing the delay gives a busy service time to recover. The ceiling keeps individual waits bounded; a separate budget bounds the number of attempts.
>
> Five retries wait for a total of 3100 milliseconds, excluding the time spent performing the operation itself.

> **Advice**
>
> For many clients, add random variation to the delay. Otherwise, clients that fail together may also retry together and create another burst of traffic.

> **Warning**
>
> A timeout does not prove that an operation failed. Retrying a payment or a write can duplicate its effect unless the operation is safe to repeat.

A narrower reading column

A single decision

```
return std::min(delay * 2, cap);
```

Run the checks

```
› make test
All checks passed.
```

Code and command output

A retry policy is easier to review when its implementation and observed behavior are kept together. Source listings carry filenames; terminal sessions describe an action and its output.

retry.cpp

```cpp
#include <algorithm>
#include <chrono>

using namespace std::chrono_literals;

std::chrono::milliseconds retry_delay(unsigned retry) {
    auto delay = 100ms;
    constexpr auto cap = 1600ms;
    while (retry > 0 && delay < cap) {
        delay = std::min(delay * 2, cap);
        --retry;
    }
    return delay;
}
```

Build and inspect

```
› clang++ -std=c++17 retry.cpp -o retry
› ./retry
attempt 1: 100 ms
attempt 2: 200 ms
attempt 3: 400 ms
attempt 4: 800 ms
attempt 5: 1600 ms
```

config/retry.conf

```
initial_delay_ms = 100
maximum_delay_ms = 1600
maximum_retries = 5
```

> **Note**
>
> Listing 1 calculates a delay; it does not sleep or send a request. Keep those effects separate from the policy.

Notes that support the argument

Callouts identify the role of a passage. Their labels and accent colors help a reader distinguish context, explanation, advice, and risk without interrupting the surrounding text.

> **Note**
>
> The initial request is not a retry. A budget of five retries permits up to six attempts in total.

> **Explanation**
>
> Increasing the delay gives a busy service time to recover. The ceiling keeps individual waits bounded; a separate budget bounds the number of attempts.
>
> Five retries wait for a total of 3100 milliseconds, excluding the time spent performing the operation itself.

> **Advice**
>
> For many clients, add random variation to the delay. Otherwise, clients that fail together may also retry together and create another burst of traffic.

> **Warning**
>
> A timeout does not prove that an operation failed. Retrying a payment or a write can duplicate its effect unless the operation is safe to repeat.

A narrower reading column

A single decision

```
return std::min(delay * 2, cap);
```

Run the checks

```
› make test
All checks passed.
```